

MLH PE Fellowship Notes

Maninder (Kaurman) Kaur

August 17, 2026

Week 2: Linux Fundamentals

Linux is an OS specifically better for open applications, more versatile, and overall better privacy options. All terminal functions run on Linux commands, so let's learn that.

Scenario: System is Slow. To see why it's slow, they could use the **top** command to show summary info of list of processes or threads running on the Kernel.

Next, check the Disk IO using **iostat** that checks the input/output to see CPU utilization, number of transfer/sec, bulk reads/writes and CPU metrics.

Then, maybe they use **sar** that sees overall system performance. We could monitor things like CPU usage, Memory utilization, I/O device consumption, battery performance, etc.

Folders in Linux

/	primary directory
/bin	essential executable programs
/boot	files needed to boot system, such as kernal, boot config files, etc.
/dev	devices nodes, used to interact with hardware
/etc	system-wide config files
/home	user home directories
/lib	libraries required for /bin and /sbin
/lib64	64-bit libraries
/media	mount points for removable CDs, DVDs, etc.
/mnt	temporarily mounted filesystems
/opt	optional application software packages
/proc	virtual pseudo-filesystem that gives info about system and processes
/run	run-time variable data
/sys	virtial pseudo-filesystem that gives info about system and processes
/root	home directory of user
/sbin	essential system binaries
/srv	site-specific data
/tmp	temporary files
/usr	multi-user applications
/var	variable data that changes system operation

VPS (Virtual Private Server) that you can SSH into and work on. Reliable performance, data protection, control, etc.

Command line basics! This is where you tell the OS what to do. Remember, if you forget how a function works just do `man {function name}` or `--help`.

Moving: `cd` allows you to move between directories, `pwd` allows you to see your location, and `ls` lets you see the contents in the folder you're in.

Manipulating Files: `touch {file name}` lets you make a file, while `nano {file name}` lets you make a file and opens up an editor. `mkdir {new folder name}` makes a new directory while `rmdir` removes it. `cp` copies files, `mv` moves them, and `rm` removes them.

Viewing: `cat {file name}` prints all the contents in the file. `head` shows you the default top 10 lines, and you can add `head -n {number}` to see more or less specific. `tail` works the same, but shows you the ending.

Text Processing: `echo "message" > {file name}` allows you to print into something or simply onto the screen. `grep "find message" {file name}` returns every line that includes the message. `sed 's/{text to replace}/{replacement text}' {file name}` will replace all things from one message with another given a file name. To save the changes, add the tag `-i`. `sort {file name}` lexicographically sorts the file name from smallest to largest, and the extra argument after `| uniq` only shows distinct numbers.

Boot Sequence steps:

- BIOS(Basic I/O System)/UEFI locates and executes the boot program or boot loader
- Boot loader loads the kernel
- Kernel starts the init process (pid=1)
- Init manages system initialization, use `systemd`

BIOS contains all the code required to gain initial access to the keyboard, display screen, disk-drives, serial communications, etc. Typically placed in ROM chip (called ROM BIOS).

Almost all Linux distros use GRUB (GRand Unified Bootloader). GRUB can boot multiple OSs, network-based diskless booting, etc.

System-wide config files are generally placed in `/etc` and their subdirectories, while ultra-specific are in `home`. Only text files should be in `/etc`. Files in `/etc/sysconfig` and its subdirectories are used by many system utilities services.

`shutdown` is used to bring the system down in a secure fashion.

Week 3: Scripting + Bash

Scripting is designed to automate tasks in the Linux interface. A Shell script is essentially a text file that contains a sequence of commands for a UNIX based OS. A Python script is a plain text file with the python extension containing Python text to execute with an interpreter.

A shell is a program that takes commands from user and gives it to the OS to process, an example of CLI (Command-Line Interface). Examples include, **bash**, **zsh**, **ksh**, **tcsh**. A terminal is a program that allows you to interact with the shell.

This is what the **date** command does.

```
$ date
Thu Jul 11 14:42:19 EST 2026
```

You will always need to start your bash script, ending in extension **.sh**, with this:

```
#!/bin/bash
```

Here's an example where we can store a variable and also use the **echo** command:

```
#!/bin/bash

echo "What is your name?"
read PERSON
echo "Hello , $PERSON"
```

```
# in terminal
$ ./test.sh
What is your name?
Creed!
Hello , Creed!
```

To actually make the bash script executable, we use the following command: **chmod +x test.sh**.

Week 4: Services and Databases

2 main databases + use cases:

- Relational databases: data stored in multiple, related tables. Adhere to ACID (Atomicity, Consistency, Isolation, and Durability). Usually structure data with predefined, fixed relations EXAMPLE: Oracle, MySQL, PostgreSQL
- NoSQL (Non-Relational): data does not conform to predefined schema. Store unstructured data, or semi-structured. EXAMPLE: MongoDB

4 types of NoSQL databases:

- Doc-oriented: main unit is a document which is JSON containing different fields. Good use case would be good for data analysis and looking through records
- Columnar: main unit is a column of a table, retrieved column wise. Log-base files good use case.
- Key-value: all data has a unique key identifier. Good for dealing with cache

Relational Operators

-eq or = (string operators)	checks if value of two operands is equal or not	[\$a -eq \$b] is not true
-ne or != (string operators)	checks if value of two operands are not true	[\$a -ne \$b] is true
-gt	checks if the value of the left operand is greater than the value of the right operand	[\$a -gt \$b] is not true
-lt	checks if the value of the left operand is less than the value of the right operand	[\$a -lt \$b] is true
-ge	checks if the value of the left operand is greater than or equal to the value of the right operand	[\$a -ge \$b] is not true
-le	checks if the value of the left operand is less than or equal to the value of the right operand	[\$a -le \$b] is true
!	logical negation	[!false] is true
-o	logical OR	[1 -o 0] is true
-a	logical AND	[1 -a 0] is false
-z	checks if given string operand size is zero	[-z \$a] is not true
-n	checks if given string operand size is non-zero	[-n \$a] is not false
str	checks if str is not the empty string	[\$a] is not false

Unix File Test Operators

-b file	Checks if the file is a block special file; if yes, then the condition becomes true.	[-b \$file] is false.
-c file	Checks if file is a character special file; if yes, then the condition becomes true.	[-c \$file] is false.
-d file	Checks if the file is a directory; if yes, then the condition becomes true.	[-d \$file] is not true.
-f file	Checks if file is an ordinary file as opposed to a directory or special file; if yes, then the condition becomes true.	[-f \$file] is true.
-g file	Checks if the file has its set group ID (SGID) bit set; if yes, then the condition becomes true.	[-g \$file] is false.
-k file	Checks if the file has its sticky bit set; if yes, then the condition becomes true.	[-k \$file] is false.
-p file	Checks if the file is a named pipe; if yes, then the condition becomes true.	[-p \$file] is false.
-t file	Checks if the file descriptor is open and associated with a terminal; if yes, then the condition becomes true.	[-t \$file] is false.
-u file	Checks if the file has its Set User ID (SUID) bit set; if yes, then the condition becomes true.	[-u \$file] is false.
-r file	Checks if the file is readable; if yes, then the condition becomes true.	[-r \$file] is true.
-w file	Checks if the file is writable; if yes, then the condition becomes true.	[-w \$file] is true.
-x file	Checks if the file is executable; if yes, then the condition becomes true.	[-x \$file] is true.
-s file	Checks if the file has size greater than 0; if yes, then the condition becomes true.	[-s \$file] is true.
-e file	Checks if file exists; is true even if file is a directory but exists.	[-e \$file] is true.

- Graph: main unit contains nodes that represent entities and edges that represent relationships. Good for graph like structures and social networks.

Database management system (DBMS)

SQL (Structured Query Language) used on Relational DBMS

Cardinality - distinctiveness of info values contained in a column. High cardinality means column contains an outsized proportion of distinctive values. Low cardinality means more repeats. Mapping constraint is a data constraint that expresses the # of entities to which another entity can be related via a relationship set. For binary relationship set R on an entity A and B, there are four possible mapping cardinalities:

- One to one 1:1
- One to many 1:M
- Many to one M:1
- Many to Many M:M

Service - group of processes (known as daemons) running continuously in background. Most Linux distros use same process manager: systemd

Systemd is a system + service manager for Linux, allows parallel startup of system services at boot time, on-demand activation of daemons, and dependency based on service control logic. **.service** -> system services, **.target** -> group of systemd units. Main features:

- Socket-based activation allows parallel services to start without losing messages when restarted.
- Starts services in parallel as soon as all listening sockets are in place. Significantly reduces time required to boot system
- Only stops running services on shutdown

systemctl - command used to control and manage **systemd** services

Activate - **systemctl start foo**, deactivate - **systemctl stop foo**, restart - **systemctl restart foo**, view status - **systemctl status foo**. More commands -> **systemctl status foo**

HTTP (hypertext transfer protocol) - set of rules for sending and receiving web pages

Web request - communicative message that goes between client/web browser to servers.

HTTP Communication happens between client and server. Client makes HTTP request and gets back a HTTP response.

HTTP request line consists of Method, URL and Version

METHODS:

- GET: used to seek info from specified source.
- HEAD: only returns headers

- POST: seeks to create and make a new object
- PUT: replace fully with new section
URL: location that HTTP is referring to
VERSION: what version? HTTP/3? HTTP/2? HTTP/1.1?
HTTP request is followed by HTTP Header Structure:
- General: applied on request and response w/o affecting database body
- Request: contains info about fetched
- Response: contains location of source requested
- Entity: info about the body of resources

HTTP response has initial status line, some header lines and an entity body
Status Codes:

- 1xx: informational
- 2xx: success (200 is request was successful)
- 3xx: redirection
- 4xx: client error (404 is file not found, 400 is bad request/not in format system can read)
- 5xx: server error (500 is server error, 505 is requested HTTP version not supported)

Command: `curl http://example.org {head -silent}`: curl is name of command, -head or -I tells curl to send an HTTP request with head method. And -silent tells curl to not display progress.

Caching - supplements primary database by removing unnecessary pressure on it. Different types of caching:

- Database Integrated: managed within database engine and has built in write through capabilities. The cons are size and capabilities.
- Local: stored within application, removing network traffic. Con is that each app has its own node, thus it cannot shared with other local caches.
- Remote: stored in dedicated servers by NoSQL.

Week 5: Testing

We need to make sure our code is always working as expected. Testing is conducted to ensure systems and applications are robust to prevent issues. We also test deployment so they must make sure CI/CD(Continuous integration/delivery/deployment) pipelines are ensured.

Types of Testing:

- White Box Testing: internal structure, design and coding of software are tested to verify flow of I/O and to improve design, usability, and security. Code is visible to testers.
- Black Box Testing: functionalities of software applications are tested without having knowledge of internal code structure, implementation details, and internal paths.
- Gray Box Testing: tests software product or application with partial knowledge of the internal structure of application.

Testing Levels:

- Unit Testing: tests individual components of the software. Makes sure all the individual parts work as intended. It is ideal to do this test prior to Integration Testing. Advantages include: quality of code improved, able to find bugs easier, changing code becomes easier, easier to understand project through documentation, debugging process simplified.

```
# if this is code to calculate volume of Cube:
def cuboid_volume(l):
    return (l*l*l)

# then let's write a Unit Test!
import unittest
class TestCuboid(unittest.TestCase):
    def test_volume(self):
        self.assertEqual(cuboid_volume(2),8)
        self.assertEqual(cuboid_volume(1),1)
        self.assertEqual(cuboid_volume(0),0)
        self.assertEqual(cuboid_volume(5.5),166.375)
```

- Integration Testing: tests the combination of different units, their interactions, subsystem unity, and code compliance with the requirements. Ideal for after Unit Tests. Advantages include: simultaneous testing of different modules becomes convenient, lifesaver for a separated team, covers large amount of code. To implement it, use **pytest** library.
- System Testing: tests the system as a whole. All the modules/components are integrated in order to verify the system works as expected or not. Ideal to do before giving it off as a final product or to go back to the drawing board. Advantages include: tests end-to-end scenarios to test the system. Tests the same environments as the production environment which understands perspective and prevents issues.

Always remember to start testing early! You should test a webpage like so:

- Functionality: Test all links, test forms, and cookies are working as expected and not broken.

- Usability: test the visibility and accessibility of the webpage and make sure the user can see and interact with everything easily. Navigation, content, etc.
- Interface: test to make sure the application is sending requests correctly to the database and outputting it right. Make sure the web server is handling the in between correctly and make sure all web queries appear in database.
- Database: test the database. Any errors in executing the queries, data integrity for creating, updating and deleting data, check response time, and test retrieval.
- Compatibility: make sure the webpage displays correctly across all devices and browsers.
- Performance: test response times and different connection speeds, load test the application and normal and peak loads, stress test to a breaking point and test a crash response.
- Security: test unauthorized access requests, accessing restricted files, check session that are killed for prolonged user activity, and use SSL certificates, websites should restrict to encrypted SSL pages.
- Crowd: get a large number of people(crowd) to execute tests.

There are also different types of tests to ensure all system, applications, environments and infrastructure are functioning as expected.

- Canary Releases: reduces risk of introducing new software version by slowly rolling out the change to a small subset of users first.
- Load Testing: simulates high demand on a software by testing to see max operating capacity of the app, whether current infrastructure is sufficient, is it sustainable, how many users can it support.
- Feature Flagging and Dark Launches: sees which features and functions can be revealed to the user or no. They leave the features disabled, and slowly enable them.
- Application Monitoring: actively monitoring application. RRUM(Real User Monitoring) where we track real world interaction. We can also use Synthetic monitoring where it reacts from simulated users.
- Tracing: we see a single interaction go through multiple layers and see if it follows the correct path.

Test Driven Development(TDD) focuses on creating unit test cases before developing the actual code. Once new code passes test, it is refactored to an acceptable standard. There are 3 phases in TDD:

- Create precise tests that verify functionality
- Correct the code once it fails
- Refactor the code once it passes to reduce redundancies or to optimize the code.

There are many benefits of TDD including creates more optimized code, helps developer better analyze and understand client requirements and requests, makes testing new functionalities easier, test coverage is higher to reduce later failure, and enhances overall productivity.

When we run the following script:

```
$ ls -l a\_file
-rw-rw-r-- 1 coop a\_project 1601 Mar 9 15:04 a\_file
```

The part, **rw-rw-r-**, meaning owner permissions(read and write), group permissions (read and write), and other permissions (read only). The user is **coop** and the group is **a_group**

There are several types of permissions that can be given: **r** is read access, **w** is write access, and **x** is execute access. If not given, the placement is replaced by a dash. Other specialized permissions exist. **setuid** allows an executable to run with owner privileges, while **setgid** runs on group privileges.

If we want to change file permissions, we use **chmod**. Here's an example where we allows owner and world to have execute privileges and remove the group write permissions:

```
$ ls -l a\_file
-rw-rw-r-- 1 coop coop 1601 Mar 9 15:04 a\_file
$ chmod uo+x, g-w a\_file
$ ls -l a\_file
-rwxr--r-x 1 coop coop 1601 Mar 9 15:04 a\_file
```

The **u** stands for user(owner), **o** stands for other(world), **g** stands for group. You can also use octal digits for shorthand chmodding, using 3 digits for each person to give permission (user, group, world).

- 1 - if the execute permission is desired
- 2 - if the write permission is desired
- 4 - if the read permission is desired
- 5 - means read/execute
- 6 - means read/write
- 7 - means read/write/execute

Whenever we need to change file ownership, we use **sudo chown [new user] [file]** and to change group ownership we use **sudo chgrp [new group] [file]**. Only superuser can do this.

POSIX ACLs(Access Control Lists) extend the simple user, group, and world system.

Groups are defined in **/etc/group** where every line looks like this: **groupname:password:GID:user1,user2...**

- Groupname is the name of the group
- password is the password placeholder, only may be set if **/etc/gshadow** exists
- GID is the group identifier. Values 0-99 are for system groups. Values between 100-GID_MIN are considered special. Values above GID_MIN are for UPG(User Private Groups).
- Then finally a list of all the members of the group.

The following may allow you to modify group settings:

- **groupadd** - add a new group
- **groupmod** - modify a group and add new users
- **groupdel** - remove a group
- **usermod** - manage a user's group memberships

HackerRank Questions

Reverse a Linked List:

"""

Complete the 'reverse' function below.

The function is expected to return an INTEGER_SINGLY_LINKED_LIST.

The function accepts INTEGER_SINGLY_LINKED_LIST llist as parameter.

For your reference:

SinglyLinkedListNode:

int data

SinglyLinkedListNode next

"""

```
def reverse(llist):
    # Write your code here
    prev = None
    current = llist

    while current is not None:
        next_node = current.next
        current.next = prev
        prev = current
        current = next_node

    return prev

if __name__ == '__main__':
    fptr = open(os.environ['OUTPUT_PATH'], 'w')

    tests = int(input())

    for tests_itr in range(tests):
        llist_count = int(input())

        llist = SinglyLinkedList()

        for _ in range(llist_count):
            llist_item = int(input())
            llist.insert_node(llist_item)

        llist1 = reverse(llist.head)

        print_singly_linked_list(llist1, ' ', fptr)
```

```

        fptr.write( '\n' )

    fptr.close()

Cycle Detection:
"""
Complete the has_cycle function below.
For your reference:

SinglyLinkedListNode:
    int data
    SinglyLinkedListNode next
"""

def has_cycle(head):
    if head is None:
        return False

    slow = head
    fast = head

    while fast is not None and fast.next is not None:
        slow = slow.next
        fast = fast.next.next

        if slow == fast:
            return True

    return False

if __name__ == '__main__':

```

Week 6: Containers

Containers are a form of system visualization, that can be used to run anything small to large. The container has all the necessary executables, binary code, libraries and config files with dependencies required to run as expected.

Containers are useful since they are a lightweight way to build, test, and deploy applications across environment. They are scalable and efficient since they use fewer resources than traditional VMs, allowing multiple containers to run on the same host without interfering with one another, allowing streamlining.

How do containers work, however? Let's first discuss a VM (Virtual Machine), which uses a hypervisor to create independent VMs that allow you to have a separate operating system. This is called virtualization, where you create a new process. Containerization encapsulates the code within its own operating system.

There is no one definition for microservice, but they are mainly independently deployable modules. This speeds up deployment and reduces the number of necessary tests since only a single module needs to be

deployed. In our case, a deployment will replace the Docker container of an individual microservice with a new Docker container, and other microservices will not be effected.

Week 8: Continuous Integration

Github Actions allows us to create custom software dev. workflows in the repo. This enables you to include Continuous Integration(CI) and Continuous Deployment (CD).

Github Actions is good since it is built into Github, meaning we can manage everything in the same place. It also allows you to test multi-container setups by adding Docker and docker-compose files. There are also multiple CI templates. Also, free!

The main concepts of Github Actions comes down to the following:

Actions: small portable building block of workflow that combined, can create a job.

Workflow: automated process that is made up of one or more jobs triggered by an event. These workflows are defined as a YAML file in the repo.

Runner: a machine with the Github Actions runner application installer. It waits for a job, runs the job's actions, and the reports the results back to Github. Can be hosted on Github to self hosted.

Job: made up of multiple steps and runs in an instance of a VM. Can be run independently or sequentially.

Step: set of tasks that can be executed by a job.

Github Actions allows you to customize your workflows to meet the needs of your app.

Github Actions includes default environment variables for each workflow run. If you need custom ones, you set these in the YAML. In this example, we create variables `MYSQL_HOST` and `MYSQL_USER` which is connected to `app.py`

```
jobs:
  example_job:
    steps:
      - name: Connect to MySQL
        run: python app.py
        env:
          POSTGRES_HOST: 127.0.0.1
          POSTGRES_PORT: admin
        ports:
          - 3306
```

You can also run scripts and shell commands, which are then executed by assigned runner. This next example has the command `npm install -g bats` run on a runner(1) and then how to run a script as an action you can store the script in your repo and supply the path and shell type(2):

First Example

```
jobs:
  example_job:
    steps:
      - run: npm install -g bats
```

Second Example

```
jobs:
  example-job:
    steps:
      - name: Run build script
        run: ./github/scripts/build.sh
        shell: bash
```

Let's say you want to save files for later. We can save these into Github as artifacts. Artifacts are the files created when you build and test your code. This is an example of creating a file and uploading as artifact:

```
jobs:
  example-job:
    name: Save output
    steps:
      - shell: bash
        run: |
          expr 1 + 1 > output.log
      - name: Upload output file
        uses: actions/upload-artifact@v2
        with:
          name: output-log-file
          path: output.log
```

If we want to download the artifact from a separate workflow run, you can use the following action:

```
jobs:
  example-job:
    steps:
      - name: Download a single artifact
        uses: actions/download-artifact@v2
        with:
          name: output-log-file
```

There are many ways to start workflows, mostly there are many templates on Github templates. If you are not sure which actions are useful you can always search them up from suggested from Github which is unique to every repo. We start with a YAML file:

```
name: CI for Node.js Project

on:
  push:
    branches: [ main ]
  pull request:
    branches: [ main ]

jobs:
  build:
    runs-on: ubuntu-latest
```

```

name: Build and Test
steps:
- uses: actions/checkout@v2
  name: Check out repository
- uses: actions/setup-node@v1
  name: Set up Node.js
  with:
    node-version: 12
- run: |
    npm c
    npm run build
    npm test
  name: Build and Test

```

As you can see, there are several parts to the file:

name: The name of the workflow will be displayed on the repo's actions page

on: the events that occur on Github and will cause the workflow to be executed. For example, you can run your workflow on push and pull request triggers, which will let you to build your code and run your tests in the CI workflow.

jobs: a list of jobs that run as part of the workflow Each job will run independently from the others and will run in different virtual environments. Jobs have names that make them easier to be identified.

Github Actions has different triggers. You can run an action when pushing code to a repo, or when creating a new tag. When building open source packages, you may receive pull requests from users. Typically, you want to run your test suite against the changed code in PR.

When we create new workflow, the default trigger is the push event. You can extend to push and PRs:

```

name: Run Tests

on: [ push, pull_request ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
    - name: ...

```

There is a job called the Cron Job used to schedule tasks to be executed in future. Often, periodically. For most cron jobs, there are 3 events present: script is to be called or executed, command executes script on a recurring basis, the actions or output of the script depends on what the script being called does. Here's an example:

```

name: Testing Cron

on:
  push:

```

```

    branches: [ main ]
  schedule:
    - cron: "0-8-1-*-*" # executes first of the month at 8AM

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v1
      - name: Running
        run: |
          echo "Running!"

```

Now what exactly is CI and CD? Continuous Integration (CI) allows us to automate the commands and tools we need to develop. For example, we often use a linter (a software that analyzes your code and informs you of any issues). Sometimes we forget to run the linter! CI is used to avoid anything getting missed and making it into the default branch and therefore into production.

The pros of CI or CD is that there are faster feedback loops that lets you know if tests are passed, increases visibility and transparency that lets other contributors know if they should merge or pull your code since they can see the status, and it shows consistency and improved quality over time. There are several helpful tasks CI can do: linter, building the project, automating tests (unit, integration, end-to-end), performance testing, accessibility testing, stale issues, notifications, and more.

The difference between Continuous Deployment (CD) and Continuous Integration (CI) lies in what happens to code after it gets pushed. CI is used to enforce a standard to each single change that a dev brings, while CD is focused on getting that change out in production without any human work. If CI fails for the project, CD must not release it to the public.

Week 9 - Monitoring and Networking

Monitoring refers to overseeing the entire dev lifecycle, from complete and real-time view of the status of services, applications, and infrastructure across environments.

In P.E., we are expected to develop faster and test regularly. Monitoring tools are essential to expand visibility throughout this process. It allows teams to identify and respond to any degradation in expected states of the system during each phase of the pipeline, quickly and in an automated manner.

the Systems Activity Reporter command, or **sar**, is an all-purpose tool for gathering system activity and performance data and creating reports that are readable by humans.

On Linux systems, the backend to **sar** is **sadc** (system activity data collector), which accumulates the statistics. It stores information in the `/var/log/sa` directory, with a daily frequency by default, but that can be adjusted. Data collection can be started from the command line, and regular periodic collection is usually started as a cron job stored in `/etc/cron.d/sysstat`.

sar will then read the data and produce a report. Here's an example of how you would use it:

```
$ sar [ options ] [ interval ] [ count ]
```

System log files are essential for monitoring and troubleshooting. These files appear under `/var/log/`. The control of how messages are dealt with is controlled by **syslogd**, but newer systemd based systems may just use **journalctl**, but may use the previous command to cooperate with it.

Important messages are sent to both the log file and the system console window. These messages are copied to `/var/log/messages`. You can view the messages in the following manners:

both commands give the same response.

```
$ sudo tail -f /var/log/messages
$ dmesg -w
```

Main Tool Commands for Process Monitoring

TOOL	PURPOSE
top	Process activity, dynamically updated
uptime	How long the system is running and the average load
ps	Detailed information about processes
pstree	A tree of processes and their connections
mpstat	Multiple processor usage
iostat	CPU utilization and I/O statistics
sar	Display and collect information about system activity
numastat	Information about NUMA (Non-Uniform Memory Architecture)
strace	Information about all system calls a process makes

There are also many tools for monitoring, debugging and tuning a system's behavior with regard to its memory. This can be complex. Memory usage and I/O throughput are related, and most memory is being used to cache the contents of a files on disk. Changing memory parameters can have a large effect on I/O performance, and changing I/O parameters can have an equally large converse effect on the virtual memory sub-system.

Memory Monitoring Utilities

UTILITY	PURPOSE	PACKAGE
free	brief summary of memory usage	procps
vmstat	detailed virtual memory statistics and block I/O, dynamically updated	procps
pmap	process memory map	procps
strace	Information about all system calls a process makes	

`/proc/meminfo` is an important file and contains wealth of information about how memory is used. `vmstat` is a multipurpose tool that displays information about the memory, paging, I/O, processor activity and processes. This is how the command usually goes:

```
$ vmstat [ options ] [ delay ] [ count ]
```

Prometheus is an open-source monitoring and alerting toolkit normally used to monitor Kubernetes environments. Prometheus relies on scraping information from your server, through pre-defined endpoints. Prometheus collects and stores its metrics as time-series data, i.e., metrics information is stored with the timestamp at which it was record alongside optional key-value pairs called labels. You can monitor plenty of metrics and it comes down to you. The common metrics are counter and gauge:

Counter: a cumulative number that increases over time or sets to zero but never decreases. For example, a counter can represent the number of requests your website serves.

Gauge: a single numerical number that either goes up or down. Compared to counter, it can decrease. For example, you can monitor CPU or memory usage.

Usually, running Prometheus involves configuring it to work with specific application you are developing. But right now, let's try running a simple Prometheus app with this single command:

```
$ docker run -p 9090:9090 prom/prometheus
```

Running Prometheus on itself is certainly fun, but does Prometheus record metrics of its host machine? In other words, how do you gather the data that your VPS produces for Prometheus to analyze and record. This is where we need an exporter integration, or commonly referred to as collectors. The official Prometheus collector is the node exporter.

Alertmanager is a companion service to Prometheus, which handles sending alerts based on conditions, which will be pre-defined by the engineer using alerting rules. Here's common terms:

Grouping: categorizing alerts based on their nature and putting them into a single notification. For example, there was a large outage and instead of a lot of notifications, one gets sent.

Inhibition: preventing alerts from going off if certain other alerts are already sent. For example, if something that lies at the base of the application is misbehaving, the alert for it gets sent, but no other alerts are sent.

Silences: muting particular alerts for a certain time. For example, if you know that something will not work while you are reorganizing your project, it is a good idea to silence the alert.

Another resource is Grafana, which is an open-source software to visualize and explore metrics. It is often used with Prometheus to serve as an extensive data visualization tool, but is not limited to work with just one potential source.

Grafana specifically works with time series, a continuous recording of metrics. Time series are represented by graphs, which places each metric based on what time it was recorded. Other examples include CPU and memory usage, sensor data, stock market index, and more. One can run it on the VPS like so:

```
$ docker run -d --name=grafana -p 3000:3000 grafana/grafana
```

Container advisor (cAdvisor) is a tool that lets you view resource usage and performance characteristics of your running Docker containers. It collects, aggregates, processes, and exports information about your active containers. But it also remembers container-specific parameters, historical usage, as well as some useful visualization of its history.

Disk performance problems can be strongly coupled to other factors, such as insufficient memory or inadequate network hardware and tuning. As a rule, a system can be considered I/O-bound when the CPU is found sitting idle waiting for I/O to complete, or the network is waiting to clear buffers.

However, one can be misled. What appears to be insufficient memory can result from too slow I/O: if memory buffers that are being used for reading and writing fill up, it may appear that memory is the problem, when the real problem is that buffers are not filling up or emptying out fast enough. Similarly, network transfers may be waiting for I/O to complete and cause network throughput to suffer.

I/O statistics are given: tps (I/O transactions per second; logical requests can be merged into one actual request), block read and written per unit of time, where the blocks are generally sectors of 512 bytes; and the total blocks read and written.

```
# -k option shows results in KB instead of blocks  
$ iostat -k
```

-m shows results in MB

`$ iostat -m`

-N shows by device number

`$ iostat -N`

-x shows more detailed report, including utilization percentage

`$ iostat -x`

Another useful function is **iotop** which must be run at root. It displays a table of current I/O usage and updates periodically, like **top**.

Network Traffic is the amount of data moving across a computer network at any given time. The more the number of active users, the higher is the data that is transferred across the network which inturn implies higher network traffic.

IP addresses are used to uniquely identify nodes across the internet. They are registered through ISPs (Internet Service Providers). The IP address is the number that identifies your system on the network. It comes in 2 variables:

IPv4: 32-bit address, composed of 4 octets (an octet is just 8bits, or a byte). Example: 148.114.252.10

IPv6: is a 128-bit address, composed of 8 16-bit octet pairs. Example: 2003:0db5:6123:0000:1f4f:0000:5529:fe2

Different IPv4 address types:

Unicast: an address associated with a specific host. Example: something like 140.211.169.4 or 64.254.248.193.

Network: an address whose host portion is set to all binary zeros. Example: Example: 192.168.1.0

Broadcast: an address to which each member of a particular network will listen. It will have the host portion set to all 1 bits, such as in 172.16.255.255 or 148.114.255.255 or 192.168.1.255.

Multicast: an address to which appropriately configured nodes will listen. The address 224.0.0.2 is an example of a multicast address.

Certain addresses and address ranges are reserved for special purposes.

127.x.x.x: reserved for the loopback (local system) interface, where $0 \leq x \leq 254$. Generally, 127.0.0.1

0.0.0.0: Used by systems that do not yet know their own address. Protocols like DHCP and BOOTP use this address when attempting to communicate with a server.

255.255.255.255: Generic broadcast private address, reserved for internal use. These addresses are never assigned or registered to anyone. They are generally not routable.

Different IPv6 address types:

Unicast: a packet is delivered to one interface

Multicast: a packet is delivered to multiple interfaces.

Anycast: a packet is delivered to the nearest of multiple interfaces (in terms of routing distance)

IPv4-mapped: An IPv4 address mapped to IPv6.

Historically, IP addresses are based on defined classes. Classes A, B, and C are used to distinguish a network portion of the address from a host portion of the address.

NETWORK CLASS	HIGHEST ORDER OCTET RANGE	NOTES
A	1-127	128 networks, 16,772,214 hosts per network, 127.x.x.x reserved for loopback
B	128-191	16,384 networks, 65,534 hosts per network
C	192-223	2,097,152 networks, 254 hosts per network
D	224-239	Multicast addresses
E	240-254	Reserved address range

netmask is used to determine how much of the address is used for the network portion and how much for the host portion as we have seen. It is also used to determine network and broadcast addresses.

NETWORK CLASS	DECIMAL	HEX	BINARY
A	255.0.0.0	ff:00:00:00	11111111 00000000 00000000 00000000
B	255.255.0.0	ff:ff:00:00	11111111 11111111 00000000 00000000
C	255.255.255.0	ff:ff:ff:00	11111111 11111111 11111111 00000000

The hostname is simply a label to distinguish a networked device from other nodes. Historically, this has also been called a nodename. For DNS purposes, hostnames are appended with a period (dot) and a domain name.

```
# shows the hostname
```

```
$ hostname
```

```
wally
```

```
# changing hostname using root privilege
```

```
$ sudo hostname lumpy
```

```
lumpy
```

```
$ sudo hostnamectl set--hostname lumpy
```

The **ip** command line utility used to configure, control and query interface parameters and control devices, routing, etc. It is preferred to the vulnerable **ipconfig** we discuss next. **ip** can be used for a wide variety of tasks. It can be used to display and control devices, routing, policy-based routing, and tunneling

```
# basic functionality
```

```
$ ip [ OPTIONS ] OBJECT { COMMAND | help }
```

```
$ ip [ -force ] -batch filename
```

shows information for all network interfaces

```
$ ip link show
```

shows information for the eth0 network interface, including statistics:

```
$ ip -s link show eth0
```

set the IP address for eth0

```
$ sudo ip addr add 192.168.1.7 dev eth0
```

bring interface eth0 down

```
$ sudo ip link set eth0 down
```

set MTU to 1480 bytes for interface eth0

```
$ sudo ip link set eth0 mtu 1480
```

set route to network

```
$ sudo ip route add 172.16.1.0/24 via 192.168.1.5
```

ipconfig is a system administration utility long found in UNIX-like operating systems used to configure, control, and query network interface parameters from either the command line or from system configuration scripts. It has been superseded by **ip** and some Linux distributions no longer install it by default.

The Predictable Network Interface Device Names (PNIDN) is strongly correlated with the use of **udev** and integration with **systemd**. There are 5 styles of names that devices can be given:

1. Incorporating Firmware of BIOS provided index numbers for on-board devices. Example: `enol`
2. Incorporating Firmware or BIOS provided PCI Express hotplug slot index numbers. Example: `ensl`
3. Incorporating physical and/or geographical location of the hardware connection. Example: `enp2s0`
4. Incorporating the MAC address. Example: `enx7837d1ea46da`
5. Using the old classic method. Example: `etho0`

Network Managers can manage different types of network inputs.

One such manager is **nmtui** where you can navigate with either the arrow keys or tab key. You can navigate and edit connections and also set the system hostname. However, some operations, such as this, cannot be done by normal users and you will be prompted for the root password to go forward.

The other one we will learn is **nmcli** which is the command line interface to Network Manager. You can issue direct commands, but it also has an interactive mode.

Routing is the process of selecting paths in a network along which to send network traffic. The routing table is a list of routes to other networks managed by the system. It defines to all networks and hosts, sending remote traffic to routers. To see the routing table you can use these two commands:

```
$ route -n
$ ip route
```

The default route is the way packets are sent when there is no other match in the routing table for reaching the specified network. It can be obtained dynamically using DHCP. However, it can also be manually configured(static). With `nmcli` it can be done via the following or you can modify configuration files directly:

```
$ sudo nmcli con mod virbr0 ipv4.routes 192.168.10.0/24 +ipv4.gateway 192.168.122.0
$ sudo nmcli con up virbr0
```

Static routes are used to control packet flow when there is more than one router or route. They are defined for each interface and can be either persistent or non-persistent. When the system can access more than one router, or perhaps there are multiple interfaces, it is useful to selectively control which packets go to which router. Either the `route` or `ip` command can be used to set a non-persistent route as in:

```
$ sudo ip route add 10.5.0.0/16 via 192.168.1.100
```

Computers are inherently insecure and need protection from the people who would intrude in on or attack them. This can be done to simply harm the system, deny services, or steal information. Security can be defined in terms of the system's ability to regularly do what it is supposed to do, integrity and correctness of the system, and ensuring that the system is only available to those authorized to use it.

Week 10 - Troubleshooting

Troubleshooting

A monolithic application is a single-threaded software app in which UX, service, and data access code are combined in single program. It is self-contained and independent of other computing apps. This has major drawbacks such as lack of resusability, constraints on scalability, as size of the codebase makes it difficult to navigate.

Most businesses are moving away from monolithic applications. Monolithic applications do not work well with a lot of deployments per day like bigger companies like Meta. To pinpoint a specific cause, you cannot deal with a monolithic application, where you cannot separate functions and parts.

Troubleshooting is the systematic approach to problem solve that is used to find and specify issues with software applications that encounter a deviation from their expected outputs.

The most common way to troubleshoot a technical issue is through "trial and error". However, there is a systematic approach that can be followed in the form of a basic troubleshooting framework. There includes following steps:

1. Identify the symptoms: Identify and recognise anomalies in the general and expected functioning of the application
2. Gathering and examining detailed info: Detailed logs, replication of the issue in a non-pod environment, and user reports are a good start into understanding details about the issue.
3. Hypothesize potential cases: As someone who potentially has to experience with the way system behaved before the symptoms, and someone who might a recollection of the latest changes brought to the application, you should come up with theories that explain the issue.

4. Verify hypotheses: once you have multiple hypotheses on what causes the problem, most times it is a good idea to eliminate them one by one.
5. Devise a plan to solve the problem: Having a list of ranging from the easiest hypotheses to debunk to the hardest ones, you should think of a plan to tackle the problem
6. Verify that the issue was resolved: Do not forget to verify if the issue is still there before moving on to the next item on your list.
7. Repeat as needed: this will probably not be the first time you encounter an issue that needs troubleshooting. But remember that if you can put your efforts just right, each time you face an issue will start taking less and less time.

Problem Determination and Troubleshooting

Linux is a common tool, but might not be great for us when we are solving system problems by volunteers from the Linux community. The concepts of Problem Determination is a crucial one for any Linux user, especially when business success is at stake. We need to learn how to diagnose problems in your Linux environment.

Do not confuse Problem Determination with Debugging. Debugging is about detecting and removing existing error. There are several phases of investigation.

(a) Using your own skills:

Whenever you face an issue, you must note two things: remember the exact time the problem occurred and you should collect info about your system. First one is important because some problems may only arise because there were other simultaneous events occurring in the system. The second one would keep you wary of different things that can change while the system is running. Things like CPU workload, amount of free disk space, or free memory are dynamic variables that can change in an instant. Having collected information, consider the following:

- What were you doing when the problem occurred. Were you trying to start a web server?
- Describe the problem. How do you know there was a problem?
- Anything that might have triggered the problem. While the problem is fresh in your mind, remember other things happening.
- Any evidence related to the problem. Error logs from your application, system logs(/var/log/messages), the error message on your screen, etc.

If you can reproduce the problem when you want, you can apply tools like **strace** (traces all of the system calls) or **ltrace** (traces functions that a program calls).

Do not forget to ask yourself if anything changed recently. If everything worked as expected, and now it doesn't, you already know where you can find the problem. That's why you must keep your changes in the production environment to a minimum. You do not want to introduce problems with unnecessary changes.

(b) kjna